

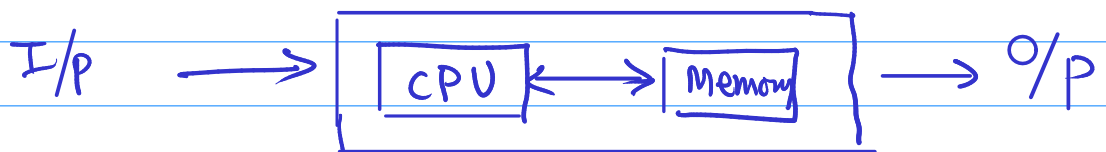
Lecture 1: Introduction to computer programming and C language

Objectives:

1. Understanding what is computer
2. Know what kind of language does computer understand
3. Understand how to give commands to computer
4. Delve into Evolution of programming languages
5. Understanding structure of C language program
6. Learn to compile and run C language program

1. What is computer?

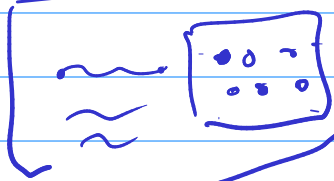
→ It is a machine with memory, computing power and some form of Input/output.



RAM - primary/temporary memory

2. What kind of language does computer understand?

→ Binary Language / machine language

Not used now a days  : Hardwire programming

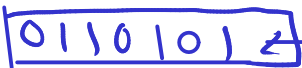
Decimal - 0-9
→ Binary - 0-1

→ Do we understand binary? → 0111011

→ we only understand numbers.

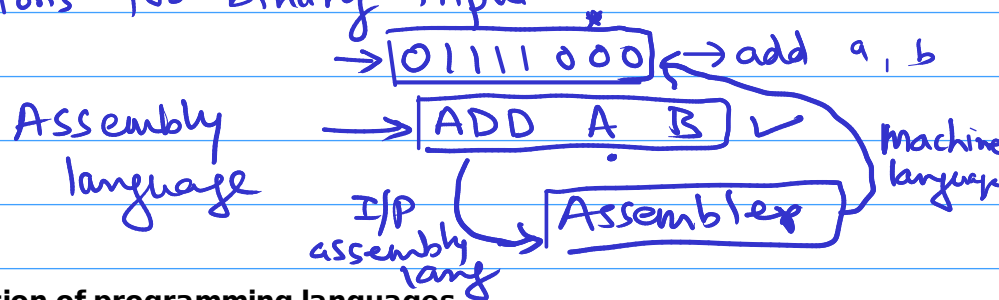
→ we do not understand binary commands.

→ → Punch card

→ → 

3. Understand how to give commands to computer

- Hardwire programming
- Punch cards
- Buttons for binary input



4. Delve into Evolution of programming languages

- Assembly programs are lengthy and harder to understand.
- Error prone
- High level language.
- More closer to real-world language compared to assembly.

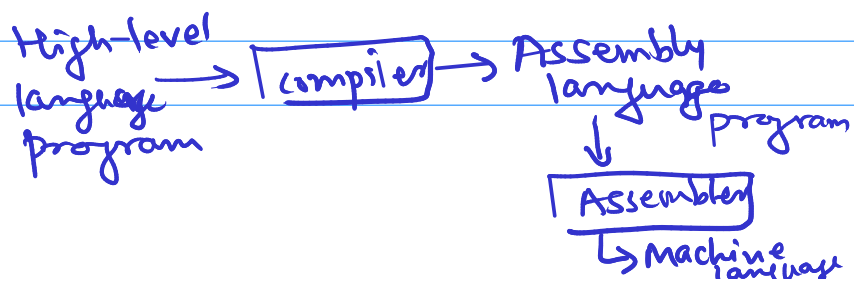
→ C, C++, java, python

one statement
`printf("Hello world!");`

- How this high level language is converted to machine level language?

→ Compiler / Interpreter

→ What is compiler?



✓ // ← single line " multiline comments → /*

*/

5. Understanding structure of C language program

Documentation section] single line Link section (header file) multi-line Ignore for now { ✓ Definition section ✓ Global declaration section . main fⁿ section (execution starts here) Ignore for now { ✓ Subprogram section	<pre> /* prog1.c */ #include <stdio.h> int main() { printf("Hello world"); return 0; } </pre>
---	---

6. Learn to compile and run C language program

```
/* Program: hello_world.c  
Author: Pandav Patel  
Date: 18 Oct 2021  
*/
```

```
#include<stdio.h>
```

```
int main() {  
    printf("Hello world");  
  
    return 0;  
}
```

-> Compile using following command:

```
gcc hello_world.c
```

-> It will generate a.out file which contains binary language program

-> We can run (execute) it using following command:

```
./a.out
```

-> Compile using following command:

```
gcc hello_world.c -o hello_world
```

-> It will generate hello_world file which contains binary language program

-> Output file name can be any valid file name and is specified after -o flag

-> We can run (execute) it using following command:

```
./hello_world
```